КОНКУРЕНТЫ

Большой сравнительный анализ EH/Unwind Analyzer на фоне мировых инструментов

Дата анализа: 1 июля 2026. Файл подготовлен для папки docs/ru проекта.

Главный вывод сразу: проект нужен не как замена IDA, Ghidra, Binary Ninja или WinDbg. Он нужен как узкий, понятный и проверяемый инструмент для одной больной зоны Windows x64: метаданные исключений и раскрутки стека, то есть .pdata, RUNTIME_FUNCTION и UNWIND_INFO. В больших инструментах эта зона обычно есть как часть огромной платформы, но редко превращена в отдельный объяснимый отчёт с 27 признаками, JSON/HTML/CSV, Ghidra-экспортом и статико-динамической связкой.

Если говорить совсем живым языком: это не «проект ради проекта». Это специализированная лупа для места, где у защитника, реверсера или исследователя вредоносного ПО часто начинается путаница. Полноценные платформы показывают весь город целиком. Ваш инструмент не пытается быть городом: он подсвечивает одну улицу, где злоумышленники, пакеры и обфускаторы могут прятать следы в технических метаданных.

Короткий вопрос	Честный ответ
Нужен ли проект в реальной работе?	Да, если задача связана с Windows PE x64, unwind-метаданными, исключениями, обработчиками, подозрительной разметкой функций или проверкой корпуса файлов.
Заменяет ли он мировые инструменты?	Нет. Он должен жить рядом с Ghidra, IDA, Binary Ninja, WinDbg, capa, YARA и PE-библиотеками.
В чём его сильная идея?	Он превращает узкую техническую тему в понятный отчёт: что сломано, почему это подозрительно, насколько уверенно, что проверять руками дальше.
Где слабое место?	Узкая платформа: PE32+ AMD64. Нет общего дизассемблера, декомпилятора, сигнатурного движка и большой базы правил как у зрелых продуктов.
Какой лучший путь развития?	Интеграция с Ghidra/IDA/Binary Ninja, экспорт в capa/YARA-процессы, больше реального корпуса, подписанные Windows-сборки и удобный релизный пакет.

1. Что именно сравниваем

EH/Unwind Analyzer анализирует Windows PE32+ AMD64-файлы через призму данных исключений и раскрутки стека. Он читает PE-заголовки, секции, exception directory, записи RUNTIME_FUNCTION и UNWIND_INFO, затем оценивает аномалии: битые диапазоны, выходы за границы, странное размещение метаданных, общий reuse unwind-записей, доминирование обработчиков, writable/exec-размещение, ошибки разбора и комбинации признаков.

В динамическом режиме проект запускает цель под Windows Debug API и собирает события, которые статикой не увидеть полностью: исключения, попадания в обработчики, VEH/VCH/UEF, RtlAddFunctionTable и RtlInstallFunctionTableCallback. Это важно: многие конкуренты умеют либо глубоко смотреть файл статически, либо мощно отлаживать процесс, но не всегда дают готовую связку «статический признак плюс runtime-подтверждение» именно по EH/unwind.

Поэтому сравнение ниже честное: рядом стоят не только прямые конкуренты, потому что прямых один-в-один почти нет. Есть большие платформы, PE-библиотеки, сигнатурные движки, отладчики и динамические фреймворки. Ваш проект попадает между ними: он уже не просто парсер, но ещё не универсальная reverse engineering IDE.

Карта рынка

Класс	Примеры	Что делают хорошо	Где остаётся зазор
Официальная база и системные утилиты	Microsoft PE/x64 EH docs, DUMPBIN, WinDbg	Дают ground truth по формату, отладке и системному поведению	Не превращают EH/unwind-аномалии в отдельную модель риска и удобный отчёт для потока файлов
Большие RE-платформы	IDA Pro, Ghidra, Binary Ninja, radare2	Дизассемблирование, графы, декомпиляция, ручной анализ, скрипты	Требуют эксперта; EH/unwind там часть огромной картины, а не готовый triage-вердикт
PE-парсеры и библиотеки	pefile, pe-parse, LIEF	Гибко читают PE, помогают строить свои пайплайны	Не дают готовую предметную оценку именно по 27 unwind/EH-сигналам
Malware triage и сигнатуры	capa, YARA, Detect It Easy	Быстро отвечают «что умеет файл», находят правила, packer/protector-семейства	Обычно не специализируются на строгой проверке RUNTIME_FUNCTION/UNWIND_INFO
Динамика и инструментирование	WinDbg, x64dbg, Frida, DynamoRIO	Можно наблюдать процесс вживую, ставить хуки, трассировать и отлаживать	Требуют настройки и не всегда дают переносимый JSON/HTML-отчёт с привязкой к статическим unwind-признакам

2. Главные конкуренты и соседи по экосистеме

Ниже не рейтинг «кто лучше вообще». У IDA и Ghidra другая весовая категория, а у YARA и capa другая задача. Полезнее смотреть так: где ваш проект реально помогает, а где разумнее не спорить с зрелым инструментом, а встроиться рядом с ним.

Microsoft PE Format, x64 Exception Handling, DUMPBIN, WinDbg

Официальная база

Что это. Это не конкурент в коммерческом смысле, а фундамент. Документация Microsoft описывает PE-формат, .pdata/.xdata, RUNTIME_FUNCTION, UNWIND_INFO и поведение раскрутки стека. DUMPBIN умеет показывать низкоуровневые данные PE/COFF, а WinDbg даёт живую отладку и crash/live-анализ.

Сильные стороны. максимально авторитетный источник по Windows ABI; WinDbg силён при анализе аварий, памяти, регистров, dump-файлов и Time Travel Debugging; DUMPBIN полезен как быстрая проверка формата и секций.

Слабые стороны в сравнении с вашим проектом. это набор инструментов, а не готовая предметная оценка EH/unwind-рисков; много ручной работы и мало объяснимого triage-вердикта; нет вашего JSON/HTML/Ghidra CSV-потока именно под 27 признаков.

Место вашего проекта. Проект стоит поверх официальной модели и автоматизирует то, что иначе аналитик собирал бы руками из спецификаций, dumpbin-вывода и отладчика.

Как сочетать. Использовать Microsoft-документацию как эталон, DUMPBIN/WinDbg - как ручную перепроверку спорных случаев.

LLVM llvm-readobj / llvm-objdump

Низкоуровневые CLI

Что это. Инструменты LLVM удобны для просмотра объектных и исполняемых файлов. llvm-readobj умеет выводить unwind-информацию и PE/COFF-специфичные блоки.

Сильные стороны. кроссплатформенность; хороши в CI и автоматизации; полезны для raw-просмотра unwind-данных.

Слабые стороны в сравнении с вашим проектом. не дают предметную модель подозрительности; не связывают статический результат с VEH/VCH/UEF и RtlAddFunctionTable; вывод больше технический, чем аналитический.

Место вашего проекта. EH/Unwind Analyzer не спорит с LLVM как с дампером. Он добавляет слой смысла: почему конкретная unwind-картина может быть подозрительной.

Как сочетать. LLVM можно использовать как независимую сверку, а ваш инструмент - как основной отчёт для защитного анализа.

Ghidra

Большая RE-платформа

Что это. Открытая платформа реверс-инжиниринга от NSA: дизассемблер, декомпилятор, графы, скрипты, поддержка разных форматов и архитектур.

Сильные стороны. мощная бесплатная платформа для ручного анализа; декомпилятор и графы помогают понять код, а не только метаданные; скрипты позволяют встраивать свои анализаторы.

Слабые стороны в сравнении с вашим проектом. порог входа высокий; EH/unwind-аномалии не являются отдельным готовым продуктовым вердиктом; без вашего CSV-импортёра аналитику придётся вручную переносить контекст.

Место вашего проекта. Ghidra - не враг, а лучший дом для результатов проекта. Ваш инструмент быстро размечает подозрительные функции и объясняет, куда смотреть.

Как сочетать. Генерировать Ghidra CSV, ставить комментарии/bookmarks и дальше открывать код уже в Ghidra.

IDA Pro

Премиальная RE-платформа

Что это. Коммерческий стандарт индустрии: дизассемблер, декомпилятор, отладчик, сигнатуры, IDAPython, огромная экосистема.

Сильные стороны. очень зрелый анализатор с сильным декомпилятором; много форматов, процессоров и плагинов; хорош для глубокой ручной работы опытного реверсера.

Слабые стороны в сравнении с вашим проектом. дорогой и не всегда доступный для учебной или небольшой команды; не заменяет специализированную автоматическую проверку вашей матрицы признаков; требует больше ручной интерпретации EH/unwind-аномалий.

Место вашего проекта. Проект может быть преданализатором перед IDA: быстро сказать, что в sample есть необычного в unwind-метаданных, а в IDA уже разбирать код.

Как сочетать. Следующий сильный шаг - экспорт/плагин для IDA по аналогии с Ghidra CSV.

Binary Ninja

RE-платформа

Что это. Современная интерактивная платформа с декомпилятором, графами, отладкой и удобными API для автоматизации.

Сильные стороны. приятная автоматизация через Python/C++/Rust; хорошая визуализация потока управления и данных; подходит для triage и командной работы.

Слабые стороны в сравнении с вашим проектом. как и Ghidra/IDA, решает широкий класс задач, а не один узкий EH/unwind-вопрос; для вашей предметной модели понадобится отдельный плагин или импорт; нет готовой 27-признаковой оценки из коробки.

Место вашего проекта. Ваш проект может стать внешним источником разметки для Binary Ninja, особенно если добавить импорт JSON/CSV.

Как сочетать. Использовать Binary Ninja для кода, а EH/Unwind Analyzer - для подготовки списка подозрительных unwind-мест.

radare2

CLI/скриптовая RE-платформа

Что это. Мощный низкоуровневый набор инструментов для анализа, отладки, патчинга, эмуляции и скриптов. Поддерживает много архитектур и форматов, включая PE.

Сильные стороны. очень гибкий CLI и скриптинг; подходит для автоматизации и нестандартных сценариев; много возможностей в одном инструменте.

Слабые стороны в сравнении с вашим проектом. сложный интерфейс для неподготовленного пользователя; анализ EH/unwind требует ручной сборки сценариев; готового человеческого отчёта по вашей evidence matrix нет.

Место вашего проекта. Ваш инструмент выигрывает простотой результата: запустил, получил JSON/HTML, список признаков, оценку и шаги ручной проверки.

Как сочетать. radare2 можно использовать как глубокий ручной/скриптовый второй слой после triage.

pefile

Python PE-библиотека

Что это. Популярная Python-библиотека для чтения и модификации PE-файлов. Удобна в скриптах, malware pipelines и исследовательских задачах.

Сильные стороны. очень удобна для быстрой Python-автоматизации; умеет разбирать заголовки, секции, директории и предупреждать о странностях; хорошо живёт в пайплайнах массового сканирования.

Слабые стороны в сравнении с вашим проектом. это библиотека, а не готовый продуктовый анализатор с вердиктом; нужно самостоятельно писать правила, отчёты и динамическую часть; нет вашей связки с Ghidra CSV и runtime-событиями.

Место вашего проекта. Проект полезен там, где не хочется собирать собственный анализатор поверх pefile. Он уже даёт законченную модель по EH/unwind.

Как сочетать. Для будущего можно сделать режим cross-check: сравнивать собственный PE-разбор с pefile на спорных файлах.

Trail of Bits pe-parse

C/C++ PE-библиотека

Что это. Лёгкий C/C++ PE-парсер с упором на устойчивость к повреждённым и злонамеренно сформированным файлам.

Сильные стороны. хорошая инженерная идея безопасного разбора через bounded buffers; подходит для встраивания в C/C++ инструменты; сильный фокус на надёжность при malformed PE.

Слабые стороны в сравнении с вашим проектом. это базовый парсер, а не отдельная аналитическая система по unwind; не содержит вашей scoring/confidence/mitigation-модели; динамическая проверка исключений и function tables не его задача.

Место вашего проекта. Ваш проект близок по инженерной философии безопасного разбора, но добавляет предметную экспертизу и отчётность.

Как сочетать. Можно использовать pe-parse как ориентир по hardening-подходам и тестированию malformed PE.

LIEF

Мультиформатная библиотека

Что это. Библиотека для разбора и модификации ELF, PE, Mach-O, COFF и других форматов с API для C++, Python, Rust, C и Node.js.

Сильные стороны. широкий охват форматов и языков; умеет не только читать, но и модифицировать бинарные форматы; удобна для комплексных исследовательских пайплайнов.

Слабые стороны в сравнении с вашим проектом. слишком широкая для вашей узкой аналитической задачи; не даёт готового объяснимого EH/unwind-вердикта; пользователь сам строит правила, отчёты и проверку динамики.

Место вашего проекта. EH/Unwind Analyzer сильнее именно как предметный инструмент, но LIEF сильнее как универсальная инфраструктура форматов.

Как сочетать. В будущем можно использовать LIEF как второй parser backend или для сравнения результатов на сложных PE.

PE-bear

PE viewer/triage

Что это. Графический инструмент для просмотра и исследования PE-файлов, популярный у malware analysts.

Сильные стороны. быстро показывает структуру PE глазами аналитика; удобен при ручном разборе секций, директорий и подозрительных областей; полезен для malformed PE.

Слабые стороны в сравнении с вашим проектом. ручной viewer, а не автоматизированная evidence matrix; не закрывает динамическую часть; не создаёт полноценный машинно-читаемый отчёт под ваш набор признаков.

Место вашего проекта. Ваш проект лучше для массовой проверки и отчётности; PE-bear лучше для ручного подтверждения конкретной структуры.

Как сочетать. После HTML/JSON-отчёта открыть файл в PE-bear и визуально проверить секции, RVA и права доступа.

Detect It Easy

File/packer detector

Что это. Инструмент для определения типов файлов, packer/protector-компонентов, эвристического распознавания и пакетного сканирования.

Сильные стороны. быстро отвечает, чем похож файл и чем он, возможно, упакован; есть GUI, CLI и лёгкий scanner; хорош как первый фильтр в malware triage.

Слабые стороны в сравнении с вашим проектом. не является глубоким unwind-анализатором; эвристика больше про тип/упаковку/сигнатуры, чем про RUNTIME_FUNCTION-целостность; не даёт вашей статико-динамической корреляции.

Место вашего проекта. Detect It Easy говорит «похоже на packer/protector». Ваш проект может сказать «а вот как именно странно выглядят unwind-метаданные».

Как сочетать. Сначала DiE для общего портрета файла, затем EH/Unwind Analyzer для узкого EH/unwind-разбора.

capa

Capability detection

Что это. Инструмент Mandiant/FLARE для поиска возможностей программы по правилам: сетевое поведение, persistence, сервисы, криптография и т.п. Работает по исполняемым файлам и некоторым динамическим отчётам.

Сильные стороны. очень понятная модель capabilities; хорошая база правил и удобный вывод доказательств; полезен для быстрой malware triage.

Слабые стороны в сравнении с вашим проектом. по смыслу ищет поведение программы, а не целостность EH/unwind-метаданных; на packed samples качество ограничено без распаковки; не заменяет вашу проверку .pdata/.xdata.

Место вашего проекта. Проект дополняет capa: capa отвечает «что программа может делать», а EH/Unwind Analyzer - «как выглядит её exception/unwind-скелет и нет ли там подозрительной механики».

Как сочетать. В будущем полезен экспорт findings как context для capa-like отчётов или общий JSON-пайплайн.

YARA

Сигнатурные правила

Что это. Классический инструмент для классификации файлов по текстовым и бинарным шаблонам с логическими условиями.

Сильные стороны. простая и мощная модель правил; широко используется в malware research, threat hunting и SOC; хорош для массового поиска известных паттернов.

Слабые стороны в сравнении с вашим проектом. правила надо писать и сопровождать; YARA не объясняет структурную причину EH/unwind-аномалии сама по себе; это не PE/unwind parser и не динамический коллектор.

Место вашего проекта. Ваш проект может находить структурные признаки, из которых потом рождаются YARA-правила или hunting-гипотезы.

Как сочетать. Использовать YARA для известных семейств, а EH/Unwind Analyzer - для объяснимых структурных признаков, особенно на неизвестных samples.

x64dbg

Windows debugger

Что это. Открытый x64/x32 отладчик для Windows с GUI, memory map, views, plugin API, scripting, PDB, stack view и patching.

Сильные стороны. очень удобен для ручной Windows-отладки; позволяет быстро смотреть исключения, стек, память и патчить; есть плагины и скрипты.

Слабые стороны в сравнении с вашим проектом. ручная интерактивная работа, не batch-отчёт по корпусу; не даёт готовой матрицы EH/unwind-признаков; меньше подходит для воспроизводимой презентационной отчётности.

Место вашего проекта. Ваш проект быстрее отбирает подозрительные места. x64dbg потом помогает руками прожать конкретный runtime-сценарий.

Как сочетать. Запускать EH/Unwind Analyzer перед x64dbg, чтобы знать, какие обработчики, диапазоны и function tables проверять.

Frida

Dynamic instrumentation

Что это. Фреймворк динамического инструментирования: внедрение скриптов, хуки функций, трассировка и наблюдение поведения на разных ОС.

Сильные стороны. очень гибкие runtime-хуки; быстрое прототипирование наблюдений без пересборки программы; подходит для сложного динамического анализа.

Слабые стороны в сравнении с вашим проектом. нужны собственные скрипты и понимание процесса; нет из коробки вашего EH/unwind-отчёта; динамика без хорошей статической карты может быть шумной.

Место вашего проекта. Frida сильнее как конструктор наблюдений. EH/Unwind Analyzer сильнее как готовый предметный отчёт по конкретной Windows x64-структуре.

Как сочетать. Можно расширить проект Frida-скриптами для точечных хуков там, где Debug API уже показал интересные события.

DynamoRIO

Runtime code manipulation

Что это. Система динамического изменения кода и построения инструментов во время выполнения: профилирование, трассировка, fuzzing, анализ памяти и инструкций.

Сильные стороны. очень мощная база для тяжёлых динамических инструментов; поддерживает сложное runtime-наблюдение; сильна в research и performance/security tooling.

Слабые стороны в сравнении с вашим проектом. это платформа разработки инструментов, а не готовый EH/unwind analyzer; стоимость внедрения выше; для вашей задачи может быть тяжелее, чем нужно.

Место вашего проекта. Ваш проект проще и ближе к практическому triage. DynamoRIO пригодится, если захочется уйти в глубокую трассировку инструкций.

Как сочетать. Оставить Debug API как понятный MVP-слой, а DynamoRIO рассматривать как исследовательское расширение.

3. Сравнительная таблица

В таблице ниже главный критерий простой: заменяет ли инструмент EH/Unwind Analyzer именно в его нише. Ответ почти везде «нет», но это не значит, что ваш проект сильнее всех. Это значит, что он решает другой, более узкий вопрос.

Инструмент	Сильнее всего в	Слабее относительно вашего проекта	Заменяет?	Лучшее сочетание
Microsoft docs / DUMPBIN / WinDbg	Эталон формата, системная отладка, crash/live-анализ	Нет готового triage по 27 EH/unwind-признакам	Нет	Эталон и ручная проверка
LLVM readobj/objdump	Raw-вывод объектных и PE/COFF-данных, CI	Мало аналитического смысла поверх unwind-данных	Нет	Независимая сверка вывода
Ghidra	Декомпиляция, графы, бесплатная ручная RE-платформа	EH/unwind-аномалии не выделены как отдельный отчёт	Нет	Импорт CSV, bookmarks, ручной код-анализ
IDA Pro	Премиальная глубокая RE, плагины, декомпилятор	Дорого, больше ручной работы по вашей нише	Нет	Предварительный triage перед IDA
Binary Ninja	Современная RE-автоматизация и UX	Нужен импорт/плагин для вашей evidence matrix	Нет	JSON/CSV как внешний сигнал
radare2	CLI, скриптинг, патчинг, отладка	Сложнее для презентационного и объяснимого отчёта	Нет	Второй слой для экспертов
pefile	Python-автоматизация PE	Библиотека без готового EH/unwind-вердикта	Частично	Cross-check parser output
pe-parse	Устойчивый C/C++ PE-parsing	Нет scoring/report/dynamic слоя	Частично	Идеи hardening и fuzzing
LIEF	Много форматов и модификация бинарников	Не предметный EH/unwind продукт	Частично	Parser backend/cross-check
PE-bear	Ручной просмотр PE	Нет batch-аналитики и динамики	Нет	Визуальная проверка секций
Detect It Easy	Packer/protector/file-type triage	Не глубокий unwind-анализ	Нет	Первый фильтр плюс ваш анализ
capa	Capabilities и понятные malware-доказательства	Не проверяет подробно .pdata/.xdata	Нет	Поведение плюс EH/unwind-структура
YARA	Сигнатурный hunting и классификация	Нужны правила, нет структурного объяснения	Нет	Из findings делать hunting-гипотезы
x64dbg	Ручная Windows-отладка	Нет batch-отчёта по 27 признакам	Нет	Проверка конкретных runtime-событий
Frida	Гибкие runtime-хуки	Нужно писать скрипты	Нет	Расширение динамики
DynamoRIO	Тяжёлое runtime-инструментирование	Слишком низкоуровневая платформа для готового triage	Нет	Исследовательская трассировка

4. Где у проекта реальная ценность

4.1. Узкая боль, которую большие платформы не закрывают красиво

Windows x64 exception/unwind metadata - это не декоративная часть PE. Эти данные нужны системе, чтобы понимать, как раскручивать стек, где границы функций, где обработчики исключений, как восстановить регистры и как пройти через prolog/epilog. Когда эти структуры повреждены, нестандартно размещены или подозрительно переиспользуются, файл может быть просто странно собран, а может быть упакован, защищён, обфусцирован или намеренно неудобен для анализа.

Большие инструменты умеют показать много деталей, но часто оставляют аналитику задачу: сам найди, сам сопоставь, сам пойми, насколько это важно. Ваш проект делает полезное промежуточное действие: превращает набор технических признаков в человеческий отчёт. Он не говорит «это точно вредонос». Он говорит «вот конкретные места, вот сила сигнала, вот почему стоит открыть Ghidra/отладчик».

4.2. Почему 27 признаков - это рабочая модель, а не случайный список

Набор признаков выглядит убедительно потому, что он не свален в одну кучу. Там есть семейства: целостность таблицы RUNTIME_FUNCTION, целостность UNWIND_INFO, размещение метаданных, переиспользование unwind-записей, доминирование обработчиков и комбинированные случаи. Это важно для практики: один слабый признак не превращается в громкий приговор, а несколько согласованных признаков поднимают уверенность.

Сильная сторона проекта - не само число 27. Сильная сторона в том, что признаки объяснимы, имеют severity/score/confidence, ослабляющие правила и manual validation steps. Это взрослее, чем просто вывести «suspicious» красным цветом.

4.3. Статико-динамическая связка

Особенно полезная часть - связь статических признаков с runtime-событиями: VEH/VCH/UEF, попадания в обработчики, регистрации динамических таблиц функций. Это уже не просто чтение файла на диске. Это попытка увидеть, оживает ли подозрительная структура во время выполнения. Для защиты и реверса такая связка ценнее одиночного дампа.

4.4. Машиночитаемость

JSON Schema, HTML, CSV и Ghidra CSV - это не украшение. Это то, что делает проект применимым в реальной работе: можно гонять корпус файлов, сравнивать отчёты, проверять семантические инварианты, импортировать комментарии в Ghidra и показывать результат на защите без ручного копирования.

5. Где проект слабее мировых решений

- Нет декомпилятора. Ghidra, IDA и Binary Ninja позволяют понять код. Ваш проект объясняет метаданные, но не заменяет анализ логики программы.
- Узкая платформа. Сейчас фокус - PE32+ AMD64. Для x86, ARM64, ELF и Mach-O нужны отдельные модели, а не простое «добавим пару if».
- Нет большой базы сигнатур. YARA, capa и Detect It Easy сильнее в известных правилах, packer/protector-семействах и capability triage.
- Динамика ограничена. Debug API-режим полезен, но это не полноценная sandbox, не полная трасса инструкций и не magic-детектор поведения.
- Нужна внешняя репутация. Для мирового уровня нужны публичные releases, signed binaries, больше реальных корпусных проверок, issues, документация adoption и плагины для популярных RE-сред.

6. Практические сценарии, где проект оправдан

Сценарий	Что даёт проект	Что добавить рядом
Malware triage PE x64	Быстро показывает, не выглядит ли exception/unwind-скелет подозрительно: битые диапазоны, странные секции, handler/reuse-комбинации	capa, YARA, DiE, Ghidra
Проверка packer/protector-гипотез	Не называет packer по имени, но показывает структурные последствия: необычное размещение, writable metadata, динамические function tables	Detect It Easy, x64dbg, Frida
Массовая проверка корпуса	JSON/Schema/HTML позволяют прогонять много файлов и быстро отделять нормальные от требующих ручного просмотра	pefile/LIEF для cross-check, SIEM/SOC scripts
Подготовка к ручному reverse engineering	CSV/Ghidra CSV размечает функции и комментарии до ручного открытия кода	Ghidra, IDA, Binary Ninja
Учебная/ВКР-защита	Хорошо показывает связь PE-формата, ABI, исключений, стека, статического и динамического анализа	Документация, демонстрационные samples, WinDbg
Внутренний quality gate	Можно проверять собственные бинарники или samples на неожиданные EH/unwind-отклонения	CI, release artifacts, signed builds

7. Итоговый вердикт

Проект нужен, но в правильной роли. Его не надо продавать как «убийцу IDA» или «полный анализатор malware». Это будет неправда и сразу ослабит впечатление. Его надо показывать как специализированный defensive tool для Windows PE x64, который быстро и объяснимо анализирует EH/unwind-метаданные, выдаёт воспроизводимые отчёты и помогает аналитику решить, куда смотреть дальше.

Если на защите или презентации спросят «зачем это, если есть Ghidra/IDA/WinDbg?», хороший ответ такой: эти инструменты позволяют всё увидеть, но не всегда быстро превращают именно unwind/EH-картину в отдельный проверяемый отчёт. Мой инструмент закрывает узкий этап triage и подготовки evidence. После него Ghidra, IDA или WinDbg становятся не заменой, а продолжением работы.

Если спросят «а нужен ли он на земле?», честно: да, нужен для специалистов, которые работают с Windows x64 PE, защитой, вредоносными samples, пакерами, обфускацией, крашами и нестандартными unwind-структурами. Массовому пользователю он не нужен. Разработчику веб-сайтов он не нужен. Но для своей узкой аудитории он имеет смысл, потому что берёт неприятную техническую тему и делает её проверяемой, объяснимой и пригодной для пайплайна.

Оценка зрелости

Критерий	Оценка	Комментарий
Идея	Сильная	Ниша узкая, но реальная. Прямых один-в-один конкурентов мало.
Инженерия	Хорошая	Есть безопасный static path, schema, тесты, corpus, отчёты, динамика и release-гейты.
Рыночная широта	Средняя	Проект не универсален; зато это честная специализация, а не распыление.
Удобство внедрения	Среднее	Нужны готовые signed binaries, плагины и понятные install/release flows.
Научная/ВКР-ценность	Высокая	Хорошо демонстрирует формат PE, ABI, эвристики, валидацию и эксперименты.
Коммерческий потенциал	Нишевый	Сильнее всего как компонент SOC/RE pipeline или open-source defensive tool, не как массовый продукт.

8. Что сделать, чтобы проект выглядел ещё сильнее

- Плагины. Сделать полноценный импорт в Ghidra уже есть хорошее начало; дальше просится IDA/Binary Ninja.
- Interop. Добавить экспорт кратких findings для capa/YARA-пайплайнов: не вместо них, а рядом.
- Больше калибровки. Расширить benign/malicious/protected corpus, сделать таблицы false positive/false negative и пороги по сценариям.
- Release UX. Подписанные Windows-сборки, source-only bundle, проверяемые checksums, понятная инструкция «скачал - запустил - получил отчёт».
- Сравнительный режим. Cross-check с pefile/LIEF/LLVM, чтобы спорные PE сразу показывали расхождения парсеров.
- Безопасность динамики. Готовые VM-рекомендации, preset для опасных samples, ясные предупреждения в CLI/доках.
- Расширение архитектур осторожно. ARM64 или PE32/x86 добавлять только как отдельные модели, иначе можно сломать аккуратность текущей AMD64-логики.

9. Источники

Ниже перечислены основные источники, по которым сверялась роль конкурентов. В тексте выводы пересказаны своими словами; сами инструменты сравнивались по их официальным страницам, документации и репозиториям.

Microsoft x64 exception handling: https://learn.microsoft.com/en-us/cpp/build/exception-handling-x64?view=msvc-170
Microsoft PE Format: https://learn.microsoft.com/en-us/windows/win32/debug/pe-format
Microsoft DUMPBIN reference: https://learn.microsoft.com/en-us/cpp/build/reference/dumpbin-reference?view=msvc-170
Microsoft Debugging tools for Windows / WinDbg: https://learn.microsoft.com/en-us/windows-hardware/drivers/debugger/
LLVM llvm-readobj command guide: https://llvm.org/docs/CommandGuide/llvm-readobj.html
Ghidra: https://github.com/NationalSecurityAgency/ghidra
IDA Pro: https://hex-rays.com/ida-pro/
Binary Ninja: https://binary.ninja/
radare2: https://github.com/radareorg/radare2
pefile documentation: https://pefile.readthedocs.io/en/latest/
pefile repository: https://github.com/erocarrera/pefile
Trail of Bits pe-parse: https://github.com/trailofbits/pe-parse
LIEF: https://github.com/lief-project/LIEF
PE-bear: https://github.com/hasherezade/pe-bear
Detect It Easy: https://github.com/horsicq/Detect-It-Easy
capa: https://github.com/mandiant/capa
YARA: https://github.com/VirusTotal/yara
x64dbg: https://x64dbg.com/
Frida: https://frida.re/
DynamoRIO: https://dynamorio.org/

Проектные источники: README.md, docs/evidence_matrix.md, docs/ARCHITECTURE.md, docs/PRODUCT_READINESS.md, результаты предыдущей локальной проверки и созданный документ «ПО ПРИЗНАКАМ НОВАЯ ВЕРСИЯ.pdf».

Приложение. Конкуренты и мой проект - прямое сравнение

Эта таблица нужна именно для защиты/презентации: она показывает не просто список известных инструментов, а место EH/Unwind Analyzer рядом с ними. Главная мысль: мой проект не пытается заменить весь мировой стек, он закрывает узкую и практически полезную задачу по Windows PE x64 EH/unwind-метаданным.

1. Сравнение по классам инструментов

Критерий	Мой проект: EH/Unwind Analyzer	IDA / Ghidra / Binary Ninja / radare2	pefile / pe-parse / LIEF / LLVM	capa / YARA / Detect It Easy	WinDbg / x64dbg / Frida / DynamoRIO
Главная задача	Быстро и объяснимо проверить .pdata, RUNTIME_FUNCTION, UNWIND_INFO, обработчики и связанные аномалии в PE32+ AMD64.	Глубокий ручной reverse engineering: дизассемблирование, декомпиляция, графы, скрипты, патчинг.	Надёжно читать, показывать или модифицировать бинарные форматы; строить свои пайплайны.	Быстро понять тип файла, packer/protector, известные признаки и capabilities.	Наблюдать поведение вживую, отлаживать, ставить хуки, трассировать, смотреть исключения и память.
Где сильнее мой проект	Даёт готовую предметную модель: 27 признаков, score, confidence, mitigations, manual validation steps, JSON/HTML/CSV/Ghidra CSV.	Не сильнее как IDE, но быстрее как узкий triage по EH/unwind без ручного ковыряния в огромной платформе.	Сильнее как готовый аналитический продукт: не надо самому писать правила, отчёт и динамическую корреляцию.	Сильнее в структурном объяснении unwind/EH-картины, особенно когда сигнатуры ничего уверенного не говорят.	Сильнее в воспроизводимом отчёте по статике плюс ограниченной динамической связке, без ручной настройки сложной трассы.
Где конкуренты сильнее	Узкая специализация: нет декомпилятора, общей сигнатурной базы, полного sandbox и поддержки всех архитектур.	Гораздо сильнее для понимания кода, графов, вызовов, типов, декомпиляции и ручного расследования.	Шире как инфраструктура форматов, особенно если нужно читать много типов файлов или модифицировать бинарники.	Сильнее для известных семейств, правил, hunting, общего malware triage и packer/protector-распознавания.	Сильнее для живого ручного анализа, сложных хуков, трассировки и исследования конкретного runtime-поведения.
Типичный результат	Отчёт: что именно подозрительно в EH/unwind, почему это важно, насколько уверенно, что проверять руками.	Рабочая среда для эксперта: код, графы, псевдокод, комментарии, скрипты.	Сырые или полуготовые данные для своего анализа.	Список сработавших правил, capabilities, packer/file-type выводы.	События процесса, регистры, стек, память, хуки, трассы.
Заменяет ли мой проект	Это и есть проект: специализированный слой EH/unwind triage.	Нет. Лучше использовать перед ними или вместе с импортом в Ghidra/IDA/Binary Ninja.	Частично для узкой задачи. Но библиотеки полезны как parser backend или cross-check.	Нет. Он дополняет их структурными доказательствами.	Нет. Он даёт карту интересных мест, которые потом удобно проверять динамически.
Лучшая рабочая связка	EH/Unwind Analyzer -> HTML/JSON -> Ghidra CSV -> ручная проверка.	Сначала мой triage, затем Ghidra/IDA/Binary Ninja для кода.	Сравнить парсинг спорных PE через LLVM/LIEF/pefile и мой отчёт.	DiE/capa/YARA для общего портрета, мой проект - для EH/unwind-скелета.	Мой отчёт показывает, куда ставить breakpoint/hook и какие handler/function table события проверять.

2. Сравнение с конкретными конкурентами

Конкурент	Где конкурент объективно сильнее	Где мой проект сильнее	Итоговая роль рядом с моим проектом
Ghidra	Декомпилятор, графы, интерактивный ручной анализ, много форматов и скриптов.	Быстрее выдаёт отдельный EH/unwind-отчёт и готовые комментарии/предупреждения для импорта.	Не замена. Лучший партнёр: мой проект готовит карту, Ghidra помогает понять код.
IDA Pro	Премиальный декомпилятор, зрелая экосистема, плагины, глубокий анализ опытным реверсером.	Дешевле, проще показать в ВКР, фокусируется на одной объяснимой модели признаков.	Не замена. Мой проект может быть предварительным фильтром перед IDA.
Binary Ninja	Современная RE-среда, удобные API, графы и командная работа.	Даёт уже готовую предметную оценку EH/unwind без написания своего плагина.	Не замена. Перспективно добавить импорт JSON/CSV.
radare2	Мощный CLI, скрипты, патчинг, отладка, много архитектур.	Понятнее для презентации и массового triage, меньше ручной настройки.	Не замена. Инструмент эксперта после первичного отчёта.
LLVM readobj/objdump	Надёжный raw-вывод форматов, удобно в CI и независимой сверке.	Добавляет смысл поверх raw-данных: severity, confidence, корреляция, шаги проверки.	Дополнение. LLVM - сверка, мой проект - аналитический вывод.
pefile	Быстрая Python-автоматизация PE, удобно в исследовательских скриптах.	Готовый продуктовый слой: отчёты, 27 признаков, динамика, Ghidra CSV.	Частично перекрывает только PE-reading, но не предметную аналитику.
pe-parse	Устойчивый C/C++ PE parser для malformed файлов.	Имеет собственную heuristic/evidence-модель и человекочитаемые отчёты.	Инженерный ориентир, не прямой заменитель.
LIEF	Много форматов и модификация бинарников через разные API.	Уже заточен под конкретный вопрос EH/unwind в Windows x64.	Хорош для cross-check и будущего backend, но не заменяет отчёт.
Detect It Easy	Быстро определяет file type, packer/protector, известные семейства.	Показывает, что именно странно в unwind-структурах, а не только общий ярлык.	Первый фильтр плюс мой глубокий EH/unwind-разбор.
capa	Отлично объясняет capabilities программы по правилам.	Сильнее в структуре PE exception/unwind и статико-динамической связке этой области.	Вместе дают хороший портрет: что файл умеет и как устроен его unwind-скелет.
YARA	Массовый hunting по правилам и известным бинарным/текстовым паттернам.	Находит структурные аномалии, из которых можно потом делать правила.	YARA ищет известное, мой проект помогает объяснить неизвестное.
WinDbg / x64dbg	Живая отладка, стек, память, исключения, breakpoints, ручное расследование.	Даёт заранее список подозрительных EH/unwind-мест и воспроизводимый отчёт.	Мой проект отвечает «куда смотреть», отладчик отвечает «что реально происходит».
Frida / DynamoRIO	Глубокое runtime-инструментирование, хуки, трассы, исследовательские сценарии.	Проще как готовый triage-инструмент и безопаснее по порогу входа.	Нужны для углубления динамики, если Debug API уже показал интересный случай.

Короткая формулировка для защиты: мой проект не выигрывает у мировых платформ в ширине. Он выигрывает в фокусе: берёт один сложный слой Windows PE x64 и делает по нему понятный, проверяемый, машиночитаемый и пригодный для ручного продолжения отчёт.